

PDF Utility Platform - Security & Architecture Audit

Role: Senior Full-Stack Architect and Lead Security Auditor

Scope: React/Vite frontend, Express/Multer backend, Node-to-Python processing bridge, Python PDF/document engines, Docker runtime, and temp-file lifecycle.

Executive Summary

The core architecture is workable and avoids the worst command-injection mistake because Python is spawned with argument arrays instead of shell interpolation. However, the platform is currently vulnerable to resource exhaustion and unsafe file-processing risks. The highest-risk issues are missing server-side file type validation, unbounded Python processing, no child-process timeout/concurrency control, and public static exposure of processed files.

1. Security Vulnerabilities

[HIGH] Server-side upload validation is extension-only or absent

Affected files: backend-api/middlewares/upload.js, backend-api/routes/pdfRoutes.js, frontend/src/components/DragDropZone.jsx.

Multer limits file size to 50 MB, but it does not validate MIME type, magic bytes, page count, image dimensions, or file structure. The frontend checks extensions, but client-side checks are bypassable.

Risk: an attacker can upload a renamed binary, malformed PDF, image decompression bomb, or Office ZIP bomb and force Python libraries to parse it.

```
// backend-api/middlewares/upload.js
const allowed = {
  '.pdf': ['application/pdf'],
  '.jpg': ['image/jpeg'],
  '.jpeg': ['image/jpeg'],
  '.png': ['image/png'],
  '.docx': ['application/vnd.openxmlformats-officedocument.wordprocessingml.document'],
  '.xlsx': ['application/vnd.openxmlformats-officedocument.spreadsheetml.sheet'],
  '.pptx': ['application/vnd.openxmlformats-officedocument.presentationml.presentation'],
  '.html': ['text/html'],
};

const fileFilter = (req, file, cb) => {
  const ext = path.extname(file.originalname).toLowerCase();
  if (!allowed[ext] || !allowed[ext].includes(file.mimetype)) {
    return cb(new Error('Unsupported file type: ${ext}'));
  }
  cb(null, true);
};

module.exports = multer({
  storage,
  fileFilter,
  limits: { fileSize: 50 * 1024 * 1024, files: 20, fields: 20, parts: 40 },
});
```

```
const { fileTypeFromFile } = require('file-type');

async function assertMagicType(filePath, allowedMime) {
  const detected = await fileTypeFromFile(filePath);
  if (!detected || !allowedMime.includes(detected.mime)) {
    throw new Error('Uploaded file content does not match expected type.');
```

[HIGH] PDF/image/document bombs can exhaust CPU and memory

Affected files: python-engine/modules/image_convert.py, basic_manipulation.py, doc_convert.py.

Examples: convert_from_path(file_path, dpi=150) converts all pages at once. PdfReader loops all pages. pdf2docx converts the whole document. openpyxl and python-pptx parse entire Office files.

Risk: a small compressed file can expand into thousands of pages, huge images, or massive XML payloads and crash the Hugging Face Space.

```
# python-engine/modules/limits.py
from pypdf import PdfReader
from PIL import Image

MAX_PDF_PAGES = 100
MAX_IMAGE_PIXELS = 40_000_000
Image.MAX_IMAGE_PIXELS = MAX_IMAGE_PIXELS

def assert_pdf_limits(file_path, max_pages=MAX_PDF_PAGES):
    reader = PdfReader(file_path, strict=False)
    pages = len(reader.pages)
    if pages > max_pages:
        raise ValueError(f"PDF exceeds {max_pages} page limit.")
    return reader
```

[HIGH] Child processes have no timeout, kill handling, or concurrency cap

Affected file: backend-api/controllers/pdfController.js. Each request spawns a Python process without timeout, queue, memory guard, or child error handling. A bad PDF can hang indefinitely and simultaneous users can spawn many workers.

```

const { spawn } = require('child_process');
const PQueue = require('p-queue').default;

const queue = new PQueue({ concurrency: Number(process.env.PYTHON_CONCURRENCY || 2) });

function runPython(scriptPath, payload, timeoutMs = 120000) {
  return queue.add(() => new Promise((resolve, reject) => {
    const child = spawn(getPythonPath(), [scriptPath], {
      stdio: ['pipe', 'pipe', 'pipe'],
      windowsHide: true,
    });

    const timer = setTimeout(() => {
      child.kill('SIGKILL');
      reject(new Error('Processing timed out. Try a smaller file.'));
    }, timeoutMs);

    let stdout = '';
    let stderr = '';
    child.on('error', reject);
    child.stdout.on('data', d => { stdout += d.toString(); if (stdout.length > 1048576) child.kill('SIGKILL'); });
    child.stderr.on('data', d => { stderr += d.toString(); if (stderr.length > 1048576) child.kill('SIGKILL'); });
    child.on('close', code => {
      clearTimeout(timer);
      if (code !== 0) return reject(new Error('Python processing failed.'));
      const parsed = JSON.parse(stdout);
      parsed.success ? resolve(parsed.output) : reject(new Error(parsed.error));
    });
    child.stdin.end(JSON.stringify(payload));
  }));
}

```

[MEDIUM] Public /processed directory exposes files by URL

Affected file: backend-api/server.js. Converted files are served statically. UUID names are hard to guess, but there is no ownership token, one-time access, request-level expiry, or authorization.

```

const crypto = require('crypto');
const downloadTokens = new Map();

function createDownloadToken(fileName) {
  const token = crypto.randomBytes(32).toString('hex');
  downloadTokens.set(token, { fileName, expiresAt: Date.now() + 15 * 60 * 1000 });
  return token;
}

app.get('/download/:token', (req, res) => {
  const record = downloadTokens.get(req.params.token);
  if (!record || record.expiresAt < Date.now()) {
    return res.status(404).json({ success: false, error: 'File expired.' });
  }
  const filePath = path.join(__dirname, 'processed', path.basename(record.fileName));
  res.download(filePath, record.fileName, () => {
    downloadTokens.delete(req.params.token);
    fs.unlink(filePath, () => {});
  });
});

```

[MEDIUM] CORS uses credentials without an auth model

Affected file: backend-api/server.js. credentials: true is enabled, but no cookie/session authentication is visible. This increases cross-origin risk surface for no current benefit.

```
app.use(cors({
  origin(origin, cb) {
    if (!origin || allowedOrigins.includes(origin)) return cb(null, true);
    cb(new Error('CORS origin denied'));
  },
  methods: ['GET', 'POST'],
  credentials: false,
  maxAge: 86400,
}));
```

[MEDIUM] Error responses leak internal implementation details

Affected file: backend-api/controllers/pdfController.js. Most handlers return err.message directly. Python errors can expose library internals, absolute paths, and processing details.

```
function publicError(err) {
  console.error('[Processing Error]', err);
  return 'The file could not be processed. Please verify the file type and try again.';
}

catch (err) {
  await logAnalytics('merge', filePaths.length, totalSize, 'failed', startTime, err.message, err.stack);
  cleanupFiles([...filePaths, outputPath]);
  res.status(422).json({ success: false, error: publicError(err) });
}
```

[LOW] Path traversal is mostly mitigated, but Python should enforce trusted directories

Good: Multer generates filenames and preserves only the extension. Node passes fixed script names, and spawn() does not use a shell. Remaining risk: Python trusts input/output paths from JSON, so future route changes could accidentally expose path traversal.

```
# python-engine/modules/path_guard.py
from pathlib import Path

BASE_UPLOADS = Path('/app/backend-api/uploads').resolve()
BASE_PROCESSED = Path('/app/backend-api/processed').resolve()

def require_inside(path, base):
  resolved = Path(path).resolve()
  if base not in resolved.parents and resolved != base:
    raise ValueError('Path outside allowed workspace.')
  return str(resolved)
```

2. Logical Flaws & Performance Bottlenecks

[HIGH] Cleanup is not guaranteed for outputs or partial files

Inputs are cleaned after success/failure, but partial output files are not consistently deleted on failure. If Python crashes mid-write, partial files remain until periodic cleanup. If Node crashes, cleanup only happens after restart and after the interval starts.

```

async function withJobCleanup(inputs, outputs, fn) {
  try {
    return await fn();
  } catch (err) {
    cleanUpFiles([...inputs, ...outputs]);
    throw err;
  } finally {
    cleanUpFiles(inputs);
  }
}

```

Recommended structure: create a per-job directory with uploads and outputs subdirectories, then delete the whole job directory in finally. That is safer than tracking individual temp files.

[HIGH] pdf2jpg loads all rendered pages into memory

Affected file: python-engine/modules/image_convert.py. `convert_from_path` returns a list of all page images before zipping begins, causing memory spikes on large PDFs.

```

def pdf_to_jpg(file_path, output_dir, poppler_path=None):
    reader = assert_pdf_limits(file_path)
    os.makedirs(output_dir, exist_ok=True)
    zip_path = os.path.join(output_dir, f"{safe_base(file_path)}_images.zip")

    with zipfile.ZipFile(zip_path, 'w', zipfile.ZIP_DEFLATED) as zip_file:
        for page_no in range(1, min(len(reader.pages), 100) + 1):
            pages = convert_from_path(file_path, dpi=120, first_page=page_no, last_page=page_no, thread_count=1)
            img_path = os.path.join(output_dir, f"page_{page_no}.jpg")
            pages[0].save(img_path, 'JPEG', quality=82, optimize=True)
            zip_file.write(img_path, os.path.basename(img_path))
            os.remove(img_path)
    return zip_path

```

[MEDIUM] Rate limiting is request-count based, not cost based

Affected file: backend-api/middlewares/rateLimiter.js. 100 requests/hour allows 100 x 50 MB uploads, or 50 images x 50 MB for jpg2pdf. That is too generous for CPU-heavy processing.

```

const heavyLimiter = rateLimit({
  windowMs: 60 * 60 * 1000,
  max: 10,
  standardHeaders: true,
  legacyHeaders: false,
  message: { success: false, error: 'Heavy conversion limit reached.' },
});

router.post('/pdf2jpg', heavyLimiter, upload.single('file'), pdfController.pdfToJpg);

```

[MEDIUM] Office conversion can become unsafe on Windows

Docker uses Linux, so COM should not run there. If deployed on Windows, Word/Excel/PowerPoint automation opens attacker-supplied documents and can trigger macros, external links, protected-view quirks, or COM hangs.

```

ENABLE_COM = os.getenv('ENABLE_OFFICE_COM') == 'true'

def word_to_pdf(file_path, output_path):
    if ENABLE_COM and COM_AVAILABLE:
        return convert_word_to_pdf_com(file_path, output_path)
    return word_to_pdf_portable(file_path, output_path)

```

3. Competitive Edge & Enterprise Improvements

- Job queue instead of direct request processing: use Redis + BullMQ or RabbitMQ. The API accepts upload, creates a job, and a worker processes with strict concurrency limits.
- WebSockets or Server-Sent Events for live progress: Python emits structured progress JSON, Node forwards status to the frontend.
- Content-addressed caching: hash input + options and reuse outputs briefly for repeated conversions.
- Sandboxed processing workers: run Python under a low-privilege user with memory limits, read-only source filesystem, and per-job temp directories.
- OCR/searchable PDF support: add Tesseract or OCRmyPDF for scanned PDF to searchable PDF, PDF to Word with OCR fallback, and language selection.

4. Frontend UX/UI Logic Improvements

- Add client-side file size and file-count validation that mirrors backend limits before upload.
- Show per-file validation errors instead of silently ignoring invalid extensions.
- Use Axios upload progress to show real upload percentage.
- Add chunked/resumable uploads for massive files using tus.io or S3/R2 multipart upload.
- Replace static download URLs with expiring download tokens and show an expiry countdown.
- Display estimated processing cost: detected pages, output type, and compression estimate.

```
await api.post(tool.endpoint, formData, {
  onUploadProgress: e => {
    const percent = Math.round((e.loaded / e.total) * 100);
    setStatusText('Uploading ${percent}%');
  },
});
```

Priority Remediation Plan

- 1. Add server-side MIME and magic-byte validation.
- 2. Add Python page, image, and archive expansion limits.
- 3. Add child-process timeout and concurrency queue.
- 4. Replace /processed static serving with expiring tokenized downloads.
- 5. Move each conversion into an isolated job directory and delete the directory in finally.
- 6. Add heavy-route rate limits based on conversion cost.
- 7. Add upload progress and frontend validation parity.

Conclusion: The platform has a solid foundation, but the processing boundary needs to be treated like hostile input execution. Once Python jobs are wrapped with validation, quotas, isolation, and lifecycle control, it becomes much closer to an enterprise-grade document utility.